

**Adaptive Confidence-Aware Multimodal Interface: A Space Shooter
Testbed for Robust Pose–Voice Fusion Under Uncertainty**

Multimodal Interaction Project

Andrea Melissa Almeida Ortega - 2179438
Dosmukhammed Raiymbek - 2179258
Prof. Danilo Avola , Maurizio Mancini

July 2026

I. ABSTRACT

This report describes an Adaptive Confidence-Aware Multimodal Interface — a system that lets a player control a simple Pygame space-shooter game using body movements (tracked by a webcam) and voice commands (captured by a microphone). The main contribution of this project is not the game itself, but a general method for figuring out, in real time, how reliable each input (camera or microphone) currently is, and adjusting how much the system relies on each one.

Instead of using a simple fixed rule like "if the camera stops working, switch to voice," the system keeps track of a smoothed confidence score for the camera input and switches between three modes — full control, assisted (smoothed) control, and safe fallback — using thresholds designed to avoid sudden or unstable switching. There is also a safety feature: if the camera becomes unreliable and the system hears a sudden loud sound, it can automatically react (for example, by activating a shield), even without a fully recognized voice command.

Overall, this design turns the project from a simple, fixed control scheme into a flexible, testable framework for combining unreliable inputs — a topic connected to broader research areas like adaptive interfaces and multimodal (multi-input) systems.

II. INTRODUCTION

Interactive systems increasingly rely on multiple, imperfect input channels — cameras, microphones, and other sensors — each of which can degrade unpredictably due to environmental conditions such as poor lighting, occlusion, or background noise. A common but brittle approach to combining such channels is to hardcode a priority order or a fixed confidence threshold that switches control from one modality to another. This approach does not reflect the graded, continuous nature of real-world sensor reliability, and it can produce jarring, discontinuous behavior for the user exactly when conditions are already difficult.

This project addresses that gap by building an Adaptive Confidence-Aware Multimodal Interface. Two input pipelines were implemented: a pose-tracking pipeline that uses a webcam and MediaPipe Pose to interpret the user's body lean as a steering signal, and a voice pipeline that uses a microphone, a voice-activity-detection model, and cloud speech recognition to interpret spoken commands. Each pipeline produces not only a raw action, but also a confidence score describing how much the system should currently trust that reading. An Adaptation Engine consumes both confidence signals and decides, frame by frame, how much control authority to grant to pose input, smoothly degrading it as confidence drops rather than cutting it off abruptly, and introducing a fallback safety behavior when vision becomes unreliable for a sustained period.

A simple 2D space shooter, built with Pygame, was used purely as an experimental testbed to exercise and observe this framework under realistic, interactive conditions — the game itself is not the research contribution. The remainder of this report describes the implementation of each pipeline, the design of the Adaptation Engine, and the qualitative behavior observed during testing.

III. METHODOLOGY

A. Implementation Specification

The project was implemented in Python, chosen for its mature ecosystem of computer-vision and audio-processing libraries and for the ease with which independent input pipelines can be developed and later integrated. Pygame was used to implement the testbed game, since it offers a lightweight, well-documented game loop that is straightforward to drive from external control signals rather than raw keyboard input. Architecturally, the system follows a two pipeline plus fusion structure, segregating the independent feature extraction loops before feeding their metrics into the central state machine. The full operational blueprint, tracing data ingestion from raw hardware captures to final game actions, is illustrated in (Diagram 1).

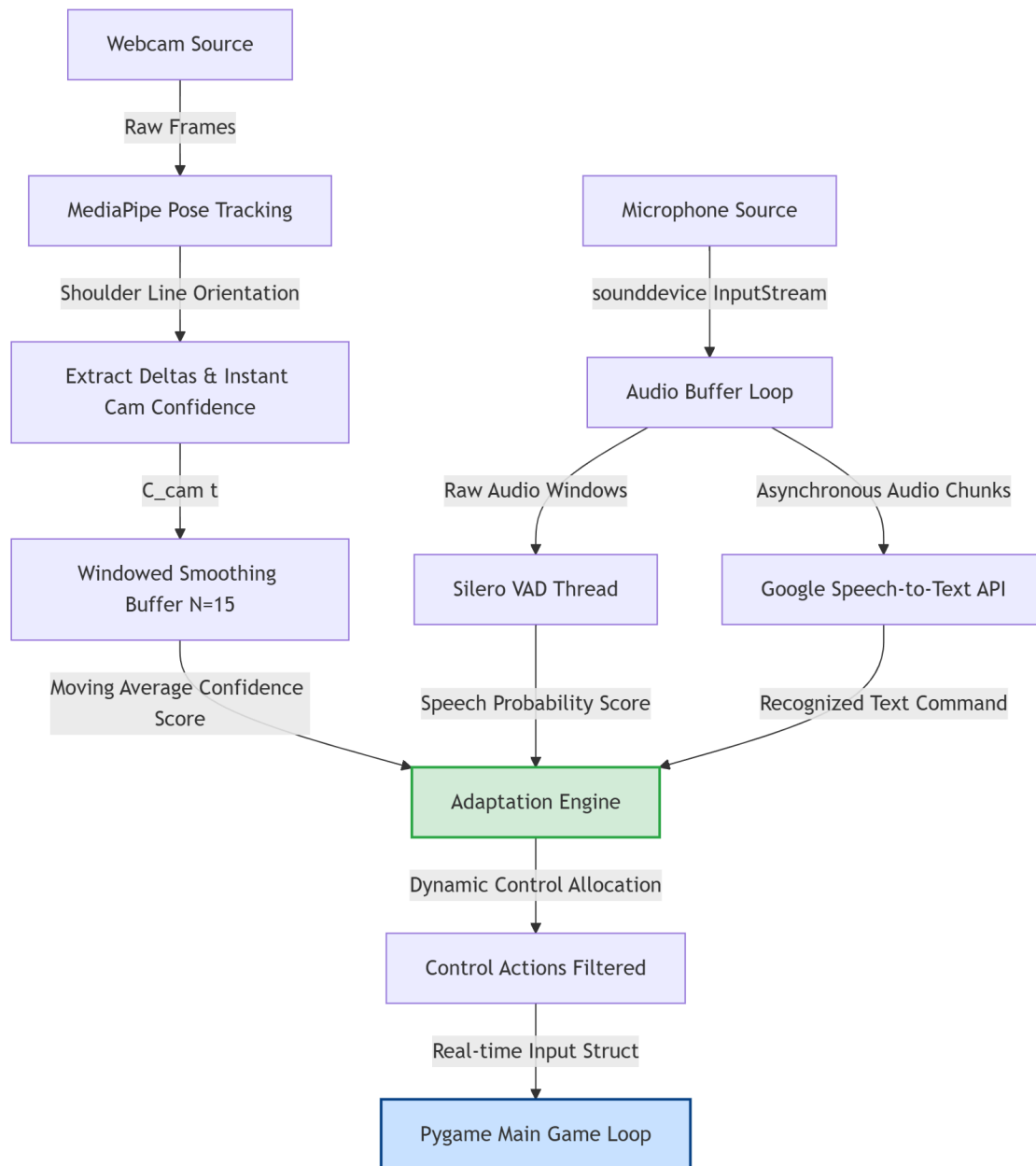


Diagram 1. High-level architecture and multimodal fusion pipeline, showcasing concurrent vision and audio processing streams feeding into the core Adaptation Engine.

For the vision pipeline, OpenCV was used to capture and display the webcam feed, and MediaPipe Pose was used to extract body landmarks in real time. For the voice pipeline, the sounddevice library was used to stream microphone audio, the Silero VAD model (loaded through PyTorch Hub) was used to estimate, per audio block, the probability that speech is present — used directly as the audio confidence score — and the SpeechRecognition library (backed by the Google Web Speech API) was used to transcribe complete utterances into text once a spoken segment had been detected. The Adaptation Engine itself

was implemented as a small, dependency-free Python module so that its logic could be reasoned about and evaluated independently of either input pipeline.

Architecturally, the system follows a two-pipeline-plus-fusion structure: the webcam feeds the pose pipeline, which outputs an action and a confidence score; the microphone feeds the voice pipeline, which outputs a command, a volume level, and a confidence score; both confidence-bearing outputs are passed into the Adaptation Engine, which resolves them into the final control signal delivered to the game.

B. Implementing Pose Detection

The pose pipeline is implemented in a PoseTracker class built around MediaPipe's Pose solution. For each incoming webcam frame, the frame is mirrored horizontally (so the player's movement feels natural) and converted to RGB before being passed to the MediaPipe pose estimator. Three landmarks are used: the nose and the left and right shoulders. Rather than using MediaPipe's internal detection score alone, to formalize this approach, let $V_i \in [0, 1]$ represent the local visibility confidence value generated by the tracking model for a given landmark i . The instantaneous vision confidence score at any given frame t , denoted as $C_{cam}(t)$, is calculated as the arithmetic mean of the visibility attributes across these three key anatomical landmarks, as defined below:

$$C_{cam}(t) = \frac{V_{nose} + V_{LSoulder} + V_{RShouder}}{3}$$

This reflects not just whether a pose was detected, but how clearly and completely the relevant body parts are currently visible to the camera.

When all three landmarks are visible above a 0.70 visibility threshold, the vertical difference between the left and right shoulder is used as a lean signal. A small threshold (0.035) is applied to this signal to classify the current frame as LEFT, RIGHT, or STATIONARY, which avoids spurious steering from natural postural jitter. Because leaning to the physical right causes the contralateral left shoulder to rise relative to the camera's coordinate space, the kinematic mapping was inverted within the application logic so that the control system reverses this delta and responds in the direction the user actually intends, rather than mirroring it (Figure 1).

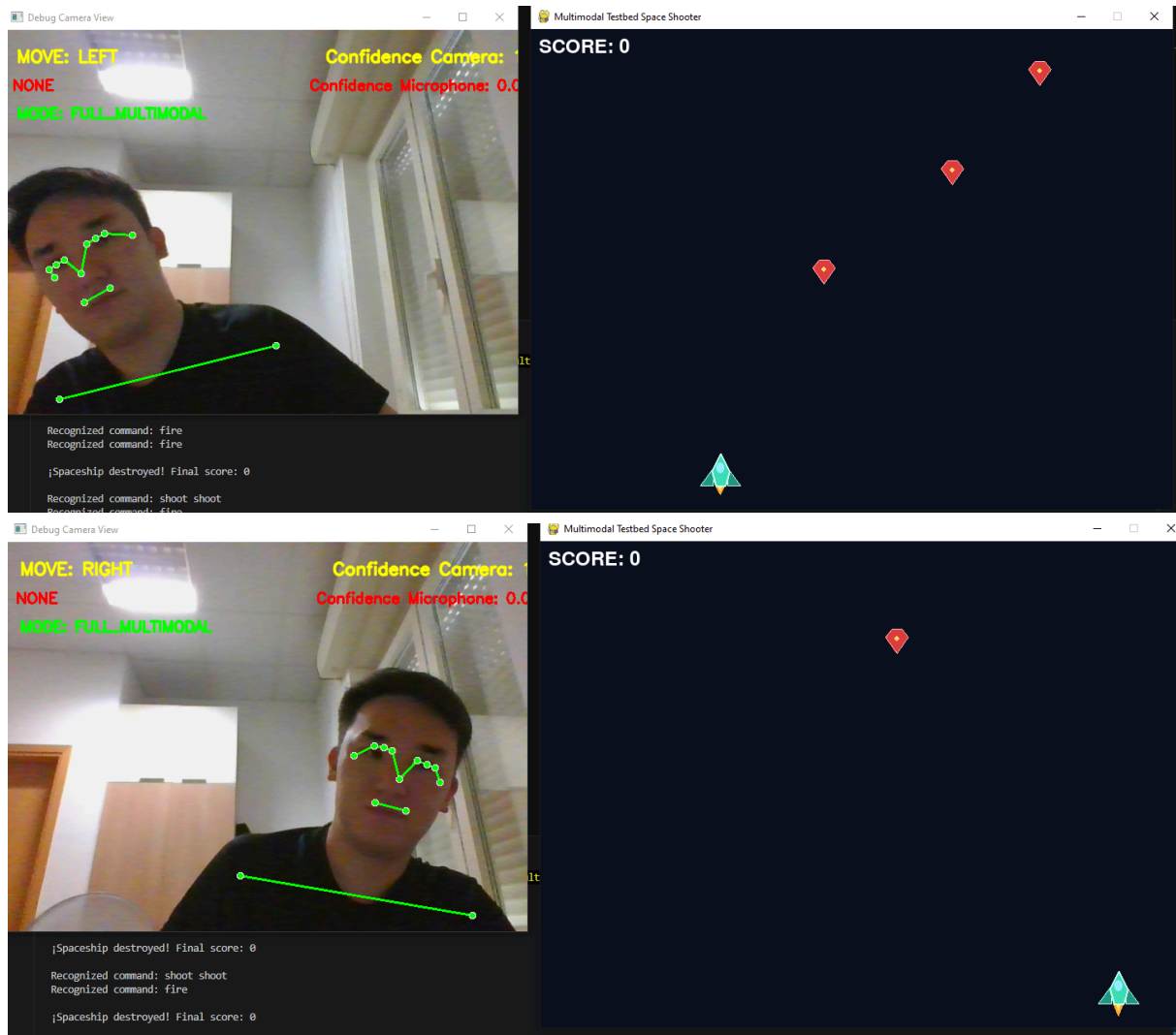


Figure 1. Body lean recognized as a LEFT (left) and a RIGHT (right) steering command, shown on the debug camera overlay.

C. Implementing Voice Control

The voice pipeline is implemented in an AudioModule class. Before gameplay begins, a short calibration routine records two seconds of silence to estimate the room's background noise floor (as an RMS value), followed by a further two seconds during which the user is asked to speak a command loudly; the loudest RMS value recorded during that segment is used to define the upper end of the volume scale. These two values allow subsequent microphone input to be normalized onto a 0–100% volume scale regardless of the user's microphone gain or room acoustics (Figure 2).

```

START AUDIO CALIBRATION
Remain completely silent for 2 seconds to measure the environment
Background noise (RMS): 0.00006

Say a command Fire / Shoot or Shield / Barrier
Maximum volume threshold (RMS): 0.11817
Successful Calibration

Volumen: [|||||||] ] 69.7% | VAD Confidence: 100.0% | Command: NONE | Recognized command: i'm just trying to do
Volumen: [ ] ] 0.0% | VAD Confidence: 0.4% | Command: NOISE / UNKNOWN | Recognized command: fire
Volumen: [ ] ] 0.0% | VAD Confidence: 0.2% | Command: NOISE / UNKNOWN | Recognized command: shield shield
Volumen: [ ] ] 0.0% | VAD Confidence: 0.2% | Command: NOISE / UNKNOWN | Recognized command: marion
Volumen: [ ] ] 0.0% | VAD Confidence: 0.2% | Command: NOISE / UNKNOWN | Recognized command: barrier
Volumen: [ ] ] 0.0% | VAD Confidence: 0.1% | Command: NOISE / UNKNOWN | Recognized command: shield shield
Volumen: [ ] ] 0.0% | VAD Confidence: 0.1% | Command: NOISE / UNKNOWN | Recognized command: should should
Volumen: [ ] ] 0.0% | VAD Confidence: 0.1% | Command: NOISE / UNKNOWN | Recognized command: fire
Volumen: [ ] ] 0.0% | VAD Confidence: 0.1% | Command: NOISE / UNKNOWN | Recognized command: yes
Volumen: [ ] ] 0.0% | VAD Confidence: 81.3% | Command: NOISE / UNKNOWN | 

```

Figure 2. Terminal output during the microphone calibration routine, measuring background noise and maximum shout volume before gameplay begins.

During gameplay, microphone audio is streamed continuously in small blocks through a sounddevice InputStream running on its own callback thread. Each block is passed to the Silero VAD model, which returns a speech probability; this value is used directly as the pipeline's audio confidence score and is also used to detect the start and end of an utterance. Rather than relying on simple signal thresholds, the local audio channel reliability is formulated as the raw probability output generated by the deep neural network model. The instant audio confidence score $C_{audio}(t)$ at time block t represents the conditional probability that a human vocalization is present within the captured audio packet:

$$C_{cam}(t) = P(\text{speech} | \text{audioBlock}(t)) \in [0, 1]$$

A short pre-roll buffer retains a small amount of audio immediately before speech is detected, and a hangover period continues to buffer audio for a few frames after speech-probability drops, so that the recognizer receives a complete utterance rather than a clipped one. Once an utterance boundary is found, the buffered audio is sent, on a background thread so as not to block real-time audio capture, to Google's speech recognition service through the SpeechRecognition library (Figure 3).

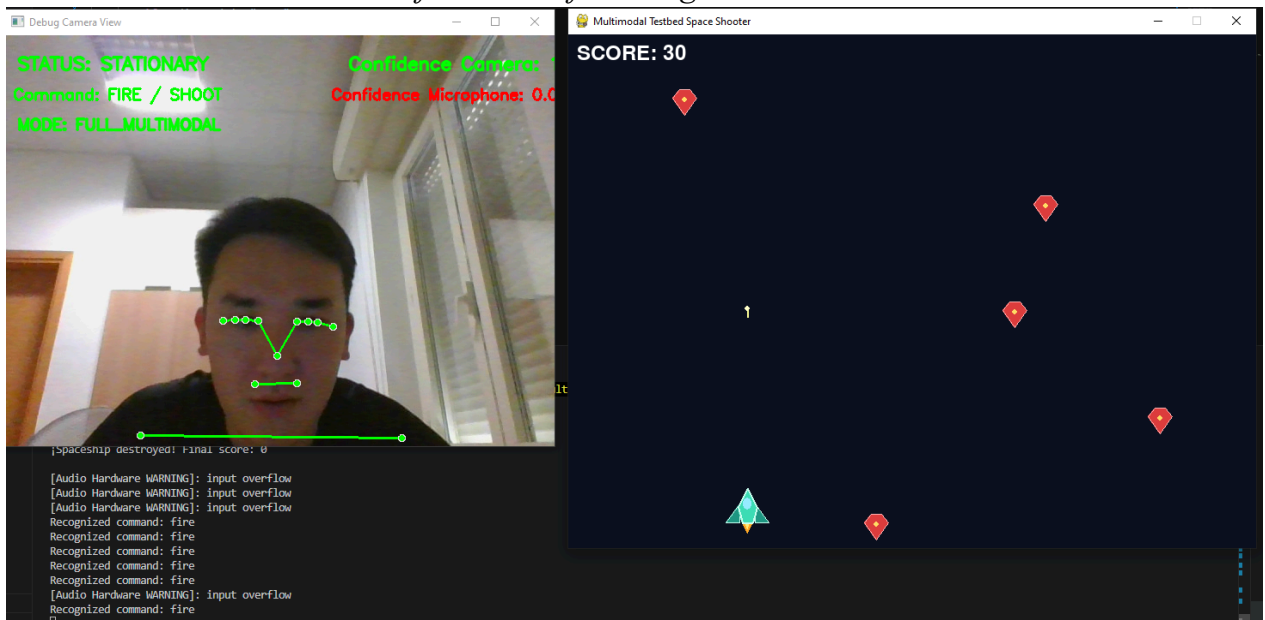
```
Background noise (RMS): 0.00006

Say a command Fire / Shoot or Shield / Barrier
Maximum volume threshold (RMS): 0.11817
Successful Calibration

Volumen: [|||||||] 69.7% VAD Confidence: 100.0% Command: NONE Recognized command: i'm just trying to do
Volumen: [ ] 0.0% VAD Confidence: 0.4% Command: NOISE / UNKNOWN Recognized command: fire
Volumen: [ ] 0.0% VAD Confidence: 0.2% Command: NOISE / UNKNOWN Recognized command: shield shield
Volumen: [ ] 0.0% VAD Confidence: 0.2% Command: NOISE / UNKNOWN Recognized command: marion
Volumen: [ ] 0.0% VAD Confidence: 0.2% Command: NOISE / UNKNOWN Recognized command: barrier
Volumen: [ ] 0.0% VAD Confidence: 0.1% Command: NOISE / UNKNOWN Recognized command: shield shield
Volumen: [ ] 0.0% VAD Confidence: 0.1% Command: NOISE / UNKNOWN Recognized command: should should
Volumen: [ ] 0.0% VAD Confidence: 0.1% Command: NOISE / UNKNOWN Recognized command: fire
Volumen: [ ] 0.0% VAD Confidence: 0.1% Command: NOISE / UNKNOWN Recognized command: yes
Volumen: [ ] 0.0% VAD Confidence: 0.4% Command: NOISE / UNKNOWN Recognized command: my photos
Volumen: [ ] 0.1% VAD Confidence: 0.9% Command: NOISE / UNKNOWN Recognized command: report
Volumen: [ ] 0.0% VAD Confidence: 0.5% Command: NOISE / UNKNOWN Recognized command: i am adding to
Volumen: [ ] 0.0% VAD Confidence: 0.2% Command: NOISE / UNKNOWN Recognized command: fire
Volumen: [ ] 0.0% VAD Confidence: 0.2% Command: Command: FIRE / SHOOT Recognized command: fire
Volumen: [ ] 0.0% VAD Confidence: 0.2% Command: NOISE / UNKNOWN Recognized command: fire
Volumen: [ ] 0.0% VAD Confidence: 0.1% Command: NOISE / UNKNOWN Recognized command: shield
Volumen: [ ] 0.0% VAD Confidence: 0.5% Command: NOISE / UNKNOWN Recognized command: please shoot shoot
Volumen: [ ] 0.0% VAD Confidence: 0.1% Command: NOISE / UNKNOWN Recognized command: shoot
Volumen: [ ] 0.0% VAD Confidence: 0.2% Command: NOISE / UNKNOWN Recognized command: i did one time you know
Volumen: [ ] 0.0% VAD Confidence: 0.5% Command: NOISE / UNKNOWN Recognized command: open the new one
Volumen: [ ] 6.8% VAD Confidence: 97.3% Command: NOISE / UNKNOWN Recognized command: play me again
Volumen: [ ] 8.3% VAD Confidence: 97.5% Command: NOISE / UNKNOWN

(venv) PS C:\Users\87055\Adaptive_Multimodal_Framework>
```

Figure 3. Debug overlay confirming that spoken audio has been captured and forwarded for recognition.



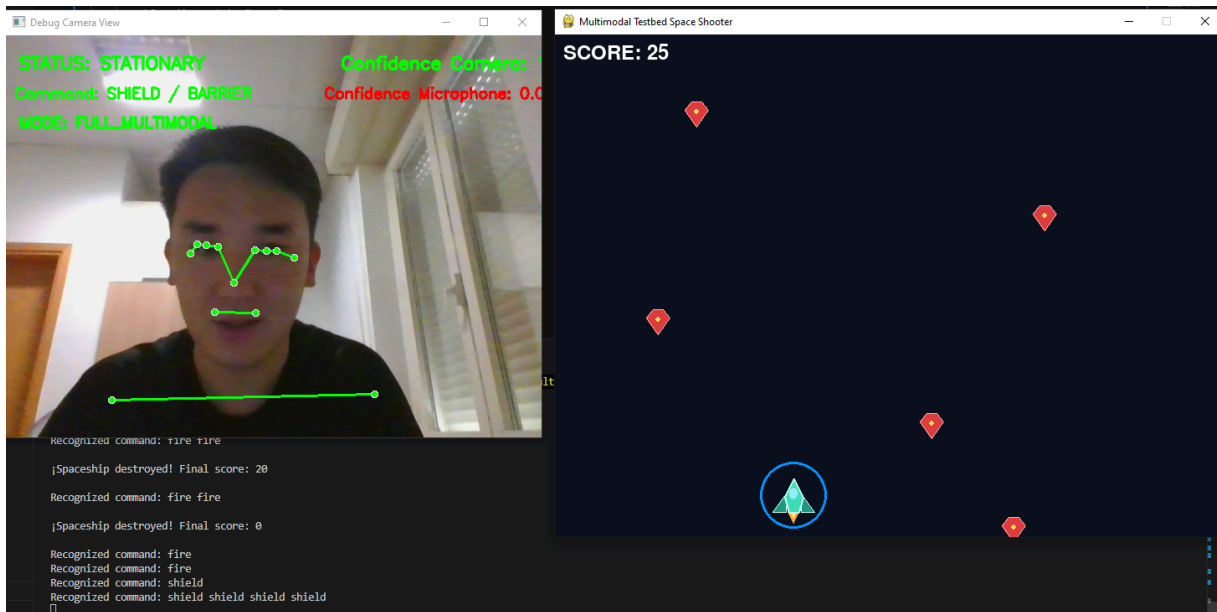


Figure 4. Debug overlay immediately after the spoken commands "fire" (left) and "shield" (right) are recognized, shown by the "Command:" line in the top-left corner of the camera view.

Recognized text is matched against an intentionally broad set of phonetic variants for two command words — "fire" (also matching "shoot," "shot," and several common misrecognitions) and "shield" (also matching "barrier" and its own common misrecognitions) — since cloud speech recognition of short, shouted single words is noticeably less reliable than recognition of full sentences. Both "shield" and its "barrier" variant are unified under the same on-screen SHIELD / BARRIER label once recognized, as visible in Figure 4 (right). If neither command is matched, the module reports either NOISE / UNKNOWN or a cloud API error, without disturbing the previously recognized command state.

During the integration phase, a concurrency bottleneck emerged when initializing the webcam pipeline and the microphone stream simultaneously. The initial computational spike from MediaPipe's model loading caused a brief frame-rate drop, which starved the audio hardware buffer and produced recurrent input-overflow warnings; the audio packets captured during this window were fragmented, dropping the Silero VAD speech confidence to zero and rendering voice commands non-functional for that period. To resolve this synchronization issue, the initialization sequences in main.py were decoupled: the system now forces a sequential boot, in which the webcam and MediaPipe tracking must be fully established and rendered before the secondary audio thread and its sounddevice callback are activated. This sequential approach

eliminated the hardware overflows and ensured clean audio packet delivery from the start.

D. Implementing the Adaptation Engine

The Adaptation Engine is the central research contribution of the project. A naive implementation might apply a single confidence threshold to decide, frame by frame, whether to trust pose input at all. In practice this produces two problems: rapid, visible flickering when confidence oscillates near the threshold, and an abrupt loss of control the instant the threshold is crossed, which feels jarring to the user and does not reflect the fact that reliability degrades gradually rather than falling off a cliff.

To address this, the engine models interaction reliability as a small finite-state machine with three modes: `FULL_MULTIMODAL`, `ASSISTED_SMOOTHING`, and `SAFE_FALLBACK`. Rather than reacting to a single frame's confidence value, the engine maintains a rolling window of the last fifteen confidence readings for the pose pipeline and bases its mode transitions on the windowed average, which absorbs momentary detection noise (for example, a single frame where a landmark briefly drops below the visibility threshold).

This filtering mechanism ensures that transient physical occlusions or short tracking dropouts do not trigger immediate, erratic state changes. The smoothed camera confidence metric $\bar{C}_{cam}(t)$ used to drive the state machine's transitions is computed over a sliding temporal window of size $N = 15$ frames, according to the following expression:

$$C_{cam}(t) = \frac{1}{N} \sum_{k=0}^{N-1} C_{cam}(t - k) \quad \text{where } N = 15$$

Transitions between modes use asymmetric hysteresis: the engine drops from `FULL_MULTIMODAL` into `ASSISTED_SMOOTHING` once average confidence falls below 0.70, and drops further into `SAFE_FALLBACK` only once it falls below 0.35 — but it does not recover back to `FULL_MULTIMODAL` until confidence climbs above a higher 0.80 threshold, and only returns from `SAFE_FALLBACK` to `ASSISTED_SMOOTHING` once confidence exceeds 0.50. Using different thresholds for degrading versus recovering prevents the system from flapping rapidly between modes when

confidence hovers near a single boundary. A minimum dwell time of 1.5 seconds in ASSISTED_SMOOTHING further prevents a brief confidence dip from causing an immediate fall into full fallback. To completely prevent state chattering under boundary optical conditions, an asymmetric hysteresis threshold model was implemented. The formal state definitions, directional transition vectors, and mathematical constraints governing this Finite State Machine (FSM) are detailed visually in (Diagram 2).

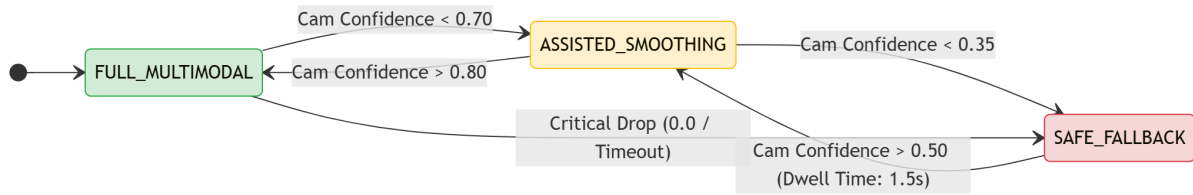


Diagram 2. Asymmetric Finite State Machine (FSM) transition logic, utilizing state hysteresis and temporal dwell windows to absorb sensor noise.

The practical effect of each mode is a change in how much authority pose input is given over the ship's horizontal movement, and how that movement is filtered. In FULL_MULTIMODAL mode, pose input drives movement directly and immediately. In ASSISTED_SMOOTHING mode, concretely, when running under this assisted control regime, the raw lean displacement signal is first bounded to reduce high-amplitude physical stress, yielding a target position $X_{target}(t)$. This target is subsequently passed through a first-order Infinite Impulse Response (IIR) filter, or Exponential Moving Average (EMA), to compute the actual ship coordinates $X_{ship}(t)$, smoothing out manual postural jitter:

$$X_{target}(t) = clamp(LeanSignal(t), -0.65, 0.65)$$

$$X_{ship}(t) = \alpha \cdot X_{target}(t) + (1 - \alpha) \cdot X_{ship}(t - 1) \quad \text{where} \quad \alpha = 0.25$$

The target movement signal is capped at 65% of full amplitude and is passed through an exponential moving average (with a smoothing factor of 0.25) before being applied, producing visibly gentler, less twitchy steering as tracking becomes less reliable. In SAFE_FALLBACK mode, pose input is no longer used for movement at all, and control falls back to the manual keyboard input already present for testing and accessibility purposes (Figure 5).

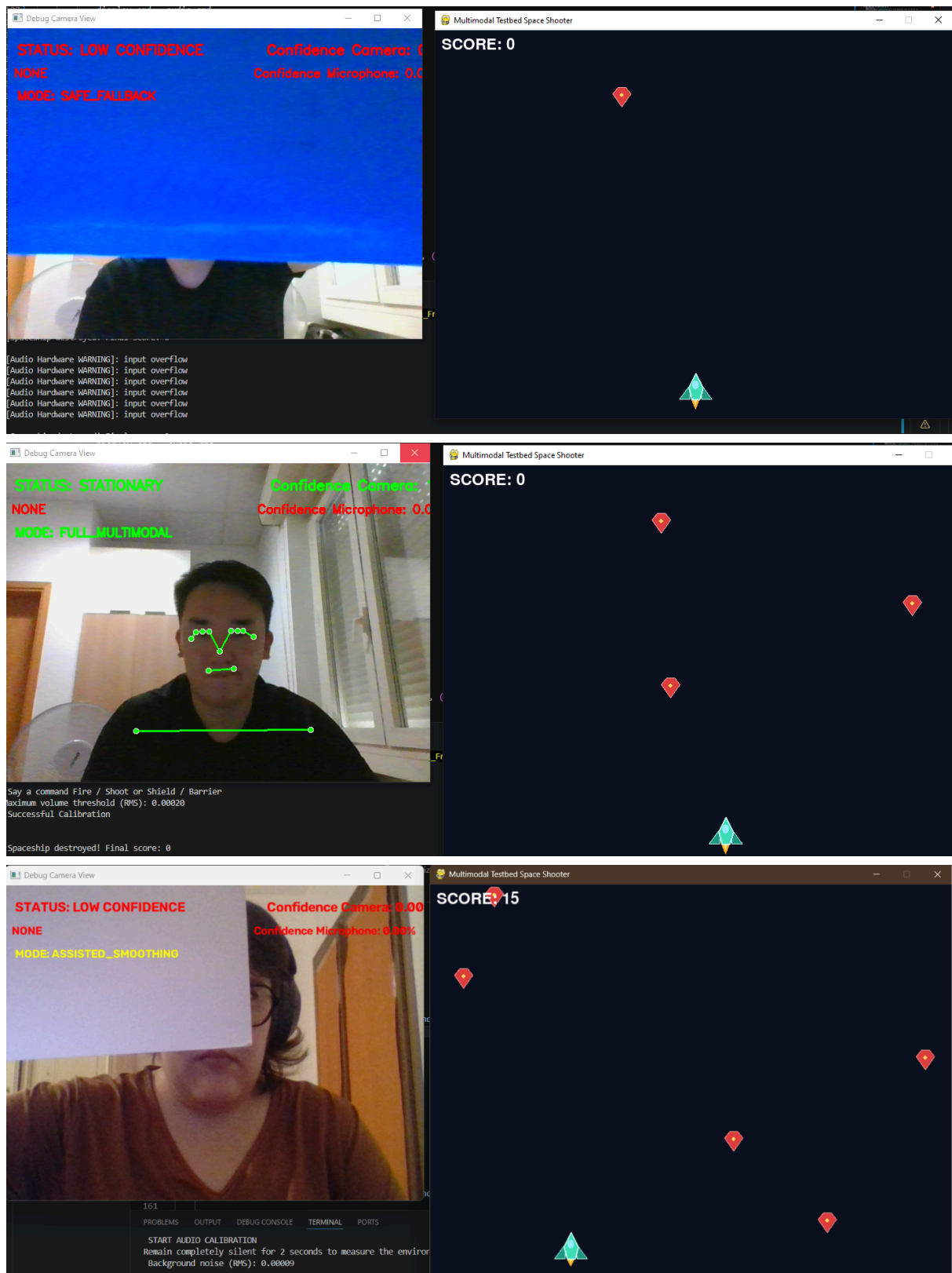


Figure 5. The three Adaptation Engine modes on the debug overlay: *SAFE_FALLBACK* (left, red), *FULL_MULTIMODAL* (middle, green), and *ASSISTED_SMOOTHING* (right, yellow).

Voice commands are handled separately from this movement state machine: a recognized FIRE or SHIELD command is applied to the game as soon as it is available and is then immediately consumed (reset to NONE) so that it is not re-applied on subsequent frames. One cross-modal safety behavior does connect the two pipelines directly: while the engine is in `SAFE_FALLBACK` mode and no voice command has just been recognized, a sudden spike in normalized microphone volume above 65% automatically raises the ship's shield. This lets an unrecognized shout or exclamation — a natural human reaction to an approaching threat when the primary visual channel has already failed — serve as an emergency compensating signal, illustrating how one modality's raw signal (loudness) can substitute for another modality's failure even without full command recognition (Figure 6).

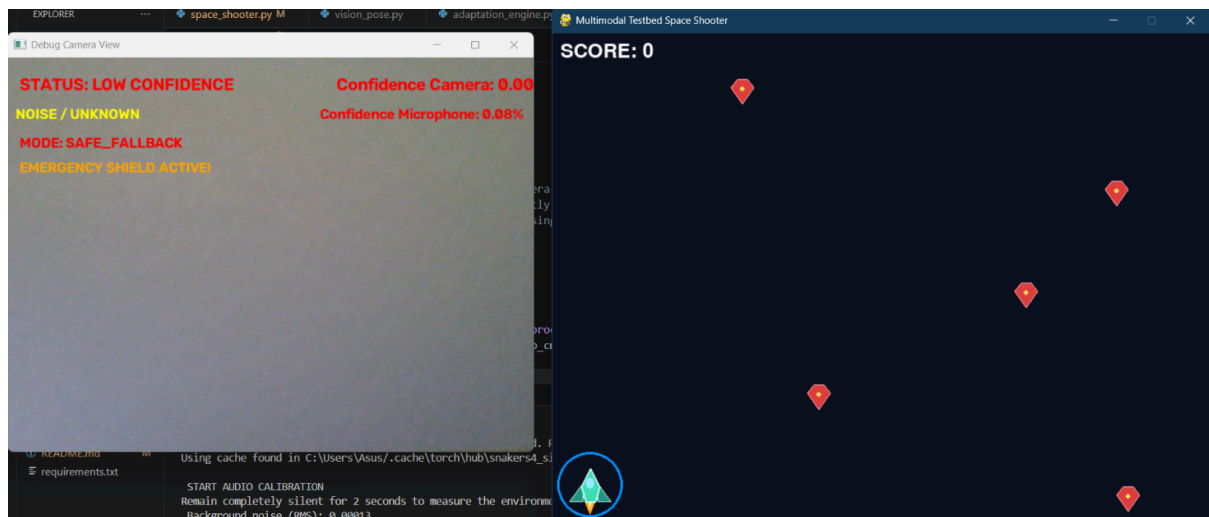


Figure 6. The automatic "EMERGENCY SHIELD ACTIVE" trigger firing in response to a loud vocalization while the engine is in `SAFE_FALLBACK` mode.

Crucially, while immediately flushing the voice command state to `NONE` ensures strict action filtering, running the environment at 60 FPS caused these visual confirmations to vanish within a single 16-millisecond frame. To provide adequate user feedback without compromising the underlying trigger logic, the UI display was decoupled from the command state using `pygame.time.get_ticks()`, artificially extending the command's visibility on the HUD for 2.0 seconds. This is paired with a dynamic color-coding system — green for successful actions, red for noise or unrecognized inputs, and white for idle states — giving both the user and evaluators clear, real-time insight into the engine's internal state.

E. Implementing the Game

The Pygame testbed is a minimal top-down space shooter used purely to exercise the Adaptation Engine under interactive conditions. The player's ship moves horizontally according to the `move_x` value produced by the Adaptation Engine, is clamped to remain within the screen bounds, and can fire bullets (limited to five simultaneous bullets) or raise a temporary shield that absorbs one collision. Enemies spawn at the top of the screen at a fixed interval and move downward; a collision with an unshielded ship ends the game and reports the final score, while a collision with a bullet removes the enemy and increases the score. Because the game logic is entirely decoupled from how `control_actions` are produced, it functioned as a convenient, low-effort testbed that did not need to change as the Adaptation Engine itself was developed and revised.

All gameplay entities are rendered as simple vector shapes built directly from Pygame's polygon, ellipse, and line primitives, rather than as external image assets, keeping the testbed self-contained and easy to modify. The player's ship is drawn as a swept-wing fighter silhouette with a rounded cockpit and a flickering engine flame whose length is randomly perturbed each frame to suggest continuous thrust; enemies are drawn as downward-pointing craft with a small glowing core; and projectiles are rendered as short glowing bolts rather than flat rectangles. This lightweight vector-art approach gives the testbed a clearer, more game-like visual identity without introducing any external asset dependencies or affecting the underlying collision and control logic (Figure 7).

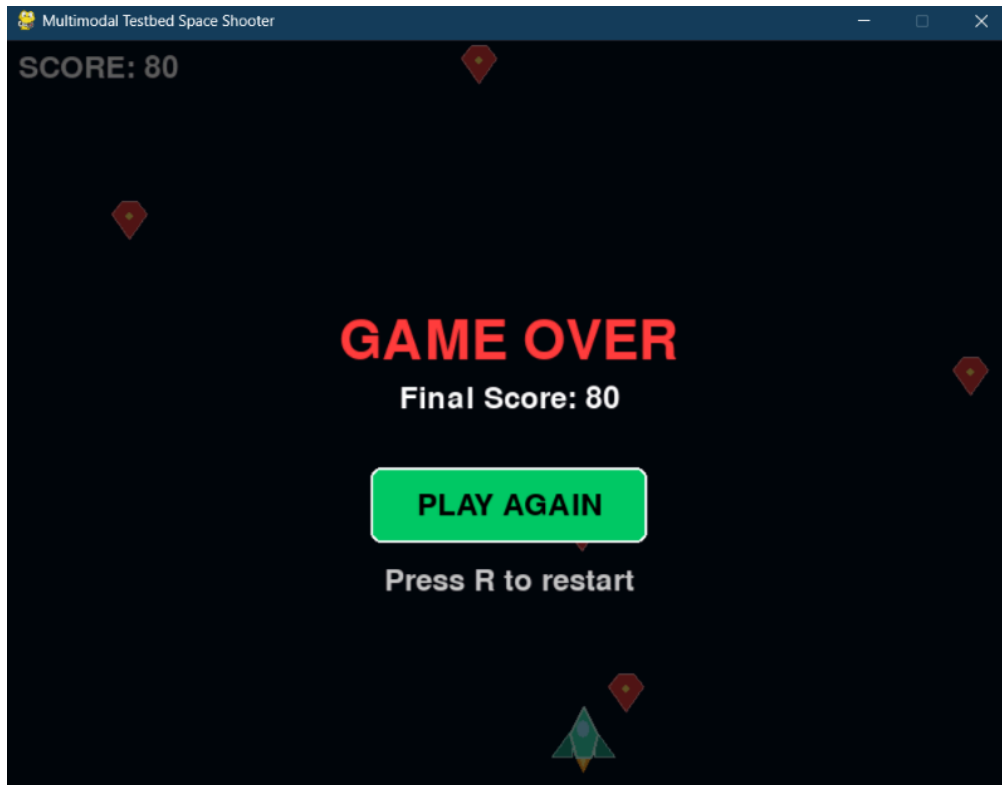


Figure 7. The Pygame space-shooter testbed window during gameplay.

To support repeated, uninterrupted testing sessions — which was especially important given that each run depends on live camera and microphone input and cannot be meaningfully scripted — the testbed was designed so that a player's death does not terminate the program. Instead, collision without an active shield transitions the game into a distinct Game Over state: gameplay freezes, a semi-transparent overlay displays the final score over the frozen scene, and the player is presented with a "Play Again" button, selectable either by mouse click or by pressing R. Restarting resets all gameplay state — score, enemies, projectiles, and ship position — while the camera feed, microphone stream, and Adaptation Engine continue running uninterrupted in the background. The program itself terminates only on an explicit user action (pressing Q/Esc or closing the window), keeping the webcam and microphone pipelines alive across many consecutive play sessions rather than requiring the script to be relaunched after every death.

IV. RESULTS AND DISCUSSION

This persistent-session design proved practically valuable during evaluation: because pose and voice calibration are performed once at startup, the Game Over / Play Again mechanism allowed many consecutive trials under varying

lighting and noise conditions to be run within a single calibrated session, rather than repeating calibration before every observation.

Qualitative testing confirmed that the intended behavior of the Adaptation Engine could be observed live during gameplay. Under normal lighting with the player fully visible to the camera, the system consistently remained in `FULL_MULTIMODAL` mode, shown in green on the debug overlay, and pose-based steering felt immediate and direct. Deliberately degrading the pose signal — by leaving the camera's field of view, occluding the shoulders and face, or moving into poor lighting — reliably drove the windowed confidence average down and produced a visible transition first to `ASSISTED_SMOOTHING` (shown in yellow, with noticeably gentler and smoother steering) and, if the degraded condition persisted, to `SAFE_FALLBACK` (shown in red, with pose-based steering fully disabled in favor of manual fallback), as illustrated earlier in Figure 5. The hysteresis-based thresholds were effective in practice: brief, momentary confidence dips did not cause the mode indicator to flicker, which was a problem observed during early testing with a naive single-threshold approach.

The cross-modal emergency shield behavior was also verified: while intentionally holding the system in `SAFE_FALLBACK` mode, an unrecognized loud vocalization reliably triggered the on-screen "EMERGENCY SHIELD ACTIVE!" indicator and raised the ship's shield, without requiring the word "shield" to be correctly recognized by the cloud speech API, as shown earlier in Figure 6.

Three practical limitations were observed. First, cloud-based speech recognition introduces a round-trip delay of roughly one to two seconds between an utterance being spoken and a command being recognized, which is acceptable for the discrete, occasional commands used here (fire, shield) but would be limiting for any modality intended to provide continuous, low-latency control. Second, because voice recognition depends on an internet connection to Google's servers, the voice pipeline's usable command recognition — as opposed to its raw confidence signal, which is computed entirely locally by Silero VAD — is not fully self-contained on the local machine. Environment setup also proved nontrivial in practice: MediaPipe, PyTorch/torchaudio, and NumPy/SciPy all impose different supported Python-version and inter-package version ranges, which required deliberate dependency pinning within a dedicated virtual environment to reach a stable, reproducible configuration.

Third, an algorithmic edge case regarding the computer-vision pipeline was identified during evaluation, which is classified here as Skeleton Compression Bias. When a user's body was systematically occluded — for example, covering up to 70% of the torso with a physical barrier — MediaPipe Pose failed to register a proportional drop in visibility or tracking confidence; instead, the tracking framework compressed the entire 33-landmark skeleton topology into the remaining 30% of the visible body area (Figure 8). This caused the camera confidence score to remain artificially high (0.94) while outputting highly distorted postural metrics, dropping sharply to 0.0 only upon complete visual loss. A related spatial anomaly occurred when a user placed a hand close to the camera lens, causing the model to mistake the hand configuration for a miniature human body and mapping hand tilt directly to ship steering. These findings highlight that visibility-based confidence heuristics alone are insufficient to intercept complex geometric tracking failures, establishing a clear motivation for incorporating structural validation in future iterations.

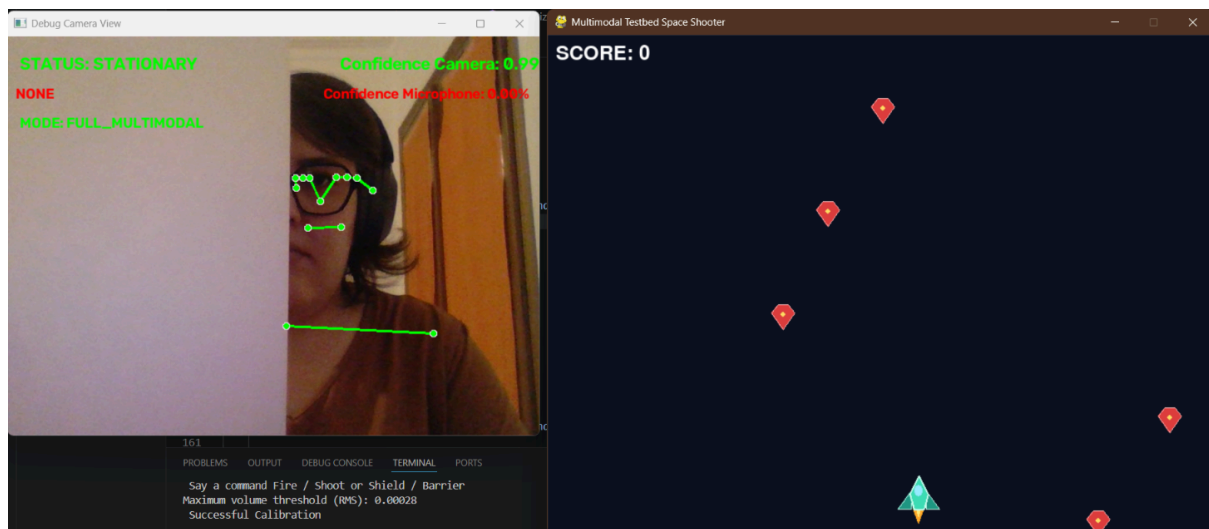


Figure 8. Skeleton Compression Bias: the debug overlay reports a deceptively high camera confidence even though the visible skeleton has collapsed into a small, distorted region of the frame after occlusion.

The current Adaptation Engine is rule-based and hand-tuned: thresholds, the smoothing factor, and the window size were chosen empirically through iterative testing rather than fitted to data. This is a reasonable and interpretable first implementation, but it also defines the clearest direction for future work: replacing or augmenting the hand-tuned thresholds with a lightweight learned model — for example, logistic regression or a small multilayer perceptron —

trained to map the two confidence signals onto an optimal blending weight, potentially adapting per user or per session. A further natural extension is a quantitative evaluation comparing this adaptive engine against a fixed-threshold baseline under controlled, reproducibly degraded conditions (e.g., systematically reduced lighting or injected background noise), measuring metrics such as steering error, mode-switch frequency, and task completion time. Finally, replacing the cloud-dependent speech recognition step with an offline recognizer would remove the framework's one remaining dependency on network connectivity.

V. CONCLUSION

This project implemented an Adaptive Confidence-Aware Multimodal Interface that fuses webcam-based pose tracking and microphone-based voice input through a confidence-driven Adaptation Engine, rather than through a fixed, hardcoded interaction rule. By modeling interaction reliability as a hysteresis-based state machine over a smoothed confidence signal, the system degrades gracefully as pose tracking becomes less reliable, and it further demonstrates a simple form of cross-modal compensation through its emergency shield behavior. A minimal Pygame space shooter served as a convenient testbed for exercising this framework interactively; the framework itself, however, is not tied to this game and could in principle be applied to any interactive system that must combine multiple uncertain input channels. The current implementation is rule-based and qualitatively validated; the clearest next steps are a learned adaptation strategy and a controlled quantitative evaluation against a fixed-threshold baseline.

VI. REFERENCES

- [1] X. Meng and S. Wei, "General Incomplete Multimodal Learning via Dynamic Quality Perception," arXiv preprint arXiv:2607.06943, 2026.
- [2] W. Wang et al., "LLM-Empowered Multimodal Fusion Framework for Autonomous Driving: Semantic Enhancement and Channel-Adaptive Design," arXiv preprint arXiv:2607.01772, 2026.
- [3] A. Bhatt and S. Mayer, "Toward a Unified Framework for Collaborative Design of Human-AI Interaction," arXiv preprint arXiv:2605.01153, 2026.
- [4] N. Shikarpur, V. Arul, and A. Huang, "The Moving Drone: Negotiating Agency Between the Voice and the Virtual," arXiv preprint arXiv:2606.13640, 2026.
- [5] Y. Li, Z. Han, H. Liu, and S. W. Su, "Dual-Agent Multiple-Model Reinforcement Learning for Event-Triggered Human-Robot Co-Adaptation in Decoupled Task Spaces," arXiv preprint arXiv:2603.06163, 2026.
- [6] J. Zheng, H. Ren, and X. Wu, "TVTA: Trajectory-Aware Viseme-Guided Temporal Aggregation for Event-Based Lip Reading," arXiv preprint arXiv:2607.08236, 2026.
- [7] E. Ramezani, M. M. Giah, M. E. Zarabadipour, A. R. Yosefian, and H. Ghadiri, "WhisperPipe: A Resource-Efficient Streaming Architecture for Real-Time Automatic Speech Recognition," arXiv preprint arXiv:2604.25611, 2026.
- [8] P. S. Wade, M. Andries, I. Kanellos, and T. Moudenc, "CBF-AFA: Chunk-Based Multi-SSL Fusion for Automatic Fluency Assessment," arXiv preprint arXiv:2506.20243, 2025.
- [9] S. Oviatt, "Ten myths of multimodal interaction," *Communications of the ACM*, vol. 42, no. 11, pp. 74-81, 1999.
- [10] C. Lugaresi et al., "MediaPipe: A framework for building perception pipelines," arXiv preprint arXiv:1906.08172, 2019.

